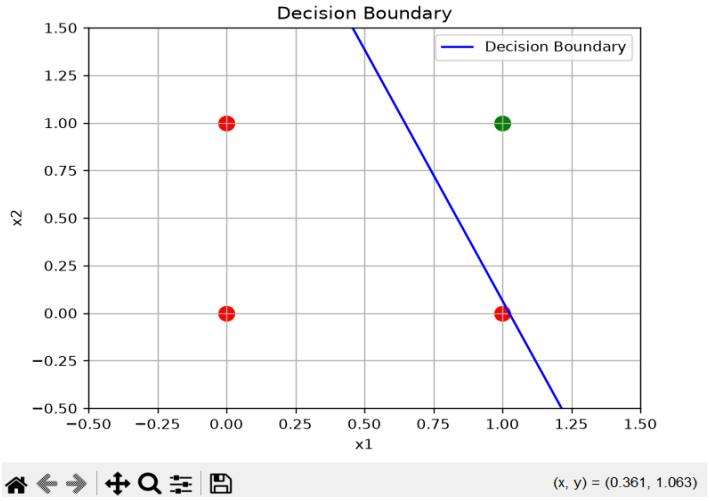
OUTPUT

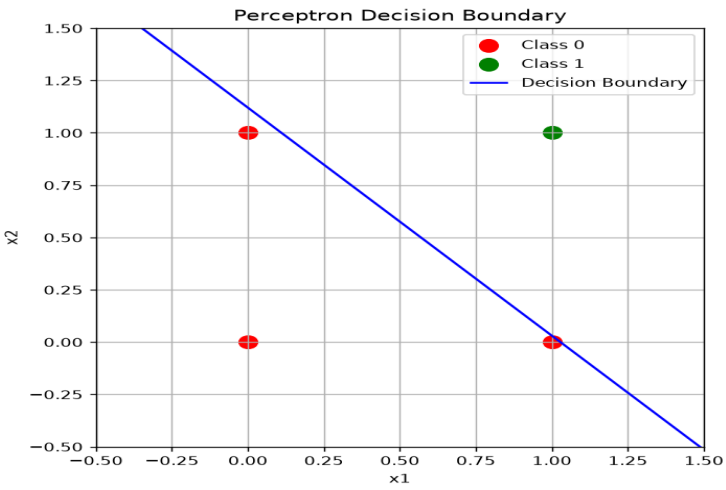
Experiment : 1.1

```
PS C:\Users\ADMIN\Downloads\NNDL-2026\Experiment-1> python experiment_1.1.py
Weight 1: [0.39015545]
Weight 2: [0.14771773]
bias [-0.4]
```



Experiment : 1.2

```
PS C:\Users\ADMIN\Downloads\NNDL-2026\Experiment-1> python experiment_1.2.py
Weights: [0.4865755  0.44646008]
Bias: -0.5
```



```python
import numpy as np
import matplotlib.pyplot as plt

x = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([0,0,0,1])

w1 = np.random.rand(1)
w2 = np.random.rand(1)
b = np.zeros(1)

lr = 0.1
epochs = 10

def step_function(z):
  if z > 0:
    return 1
  else:
    return 0

for epoch in range (epochs):
  for i in range (len(x)):
    a = w1 * x[i][0] + w2 * x[i][1] + b
    y_pred = step_function(a)

    error = y[i] - y_pred

    w1 += lr * error * x[i][0]
    w2 += lr * error * x[i][1]
    b += lr * error

print('Weight 1:',w1)
print('Weight 2:',w2)
print('bias',b)

# Decision Bundary plotting
for i in range(len(x)):
    if y[i] == 0:
        plt.scatter(x[i][0], x[i][1], color='red', s=100)
    else:
        plt.scatter(x[i][0], x[i][1], color='green', s=100)

x_vals = np.array([-0.5, 1.5])

if w2 != 0:
    y_vals = -(w1 * x_vals + b) / w2
    plt.plot(x_vals, y_vals, 'b-', label='Decision Boundary')
else:
    plt.axvline(x=-b / w1, color='blue', label='Decision Boundary')

plt.xlim(-0.5, 1.5)
plt.ylim(-0.5, 1.5)
plt.xlabel("x1")
plt.ylabel("x2")
plt.title("Decision Boundary")
plt.grid(True)
plt.legend()

plt.show()
```

```python
import numpy as np
import matplotlib.pyplot as plt

x = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([0,0,0,1])

w1 = np.random.rand(1)
w2 = np.random.rand(1)
b = np.zeros(1)

lr = 0.1
epochs = 10

def step_function(z):
  if z > 0:
    return 1
  else:
    return 0

for epoch in range (epochs):
  for i in range (len(x)):
    a = w1 * x[i][0] + w2 * x[i][1] + b
    y_pred = step_function(a)

    error = y[i] - y_pred

    w1 += lr * error * x[i][0]
    w2 += lr * error * x[i][1]
    b += lr * error

print('Weight 1:',w1)
print('Weight 2:',w2)
print('bias',b)

# Decision Bundary plotting
for i in range(len(x)):
    if y[i] == 0:
        plt.scatter(x[i][0], x[i][1], color='red', s=100)
    else:
        plt.scatter(x[i][0], x[i][1], color='green', s=100)

x_vals = np.array([-0.5, 1.5])

if w2 != 0:
    y_vals = -(w1 * x_vals + b) / w2
    plt.plot(x_vals, y_vals, 'b-', label='Decision Boundary')
else:
    plt.axvline(x=-b / w1, color='blue', label='Decision Boundary')

plt.xlim(-0.5, 1.5)
plt.ylim(-0.5, 1.5)
plt.xlabel("x1")
plt.ylabel("x2")
plt.title("Decision Boundary")
plt.grid(True)
plt.legend()

plt.show()
```